PhySA: Decoupling Aesthetic and Physical Rewards for Humanoid Locomotion

Yunwan Fu[1] **Sicheng He**[2] **Jiageng Mao**[2] **Yue Wang**[2]
[1]Shanghai Jiao Tong University [2]University of Southern California
asterrose@sjtu.edu.cn

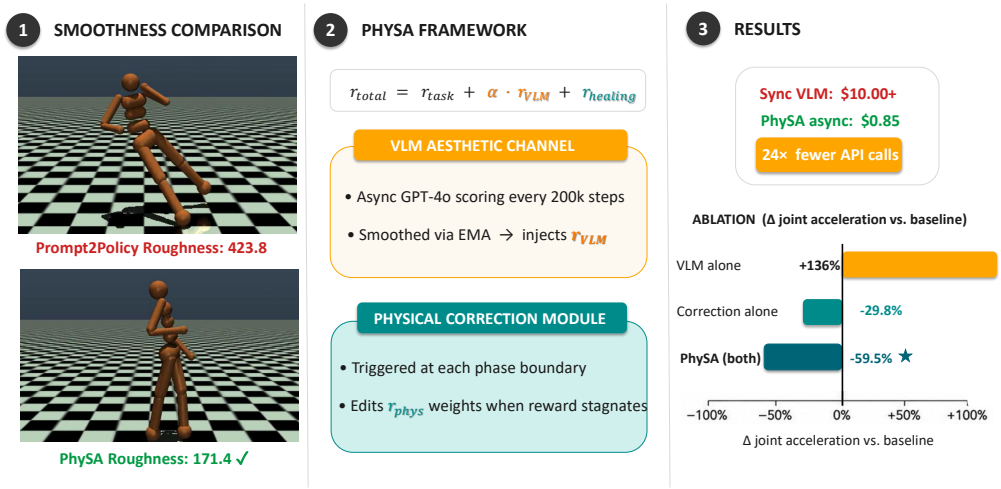


Figure 1: **Left:** In our main training run (a single run), PhySA reduces joint acceleration by 59.5% relative to Prompt2Policy at the same forward-velocity target; a three-seed replication gives a smaller, non-significant 7.2%. **Center:** The two channels pull in opposite directions: the aesthetic channel alone *raises* joint acceleration by 136%; the correction module alone reduces it by 29.8%. **Right:** PhySA issues 24× fewer VLM queries than a synchronous per-update scorer would (estimated from the number of policy updates; measured PhySA cost $0.85/10M steps); the smoothness gain does not carry over to other simulators, even after rescaling the reward weights.

Abstract: Humanoid locomotion policies trained without an explicit smoothness objective tend to produce jerky, high-energy gaits that stress actuators. Vision-language models (VLMs) offer an appealing reward source: given a rendered rollout, a VLM returns an *aesthetic quality* score on $[0, 10]$ without task-specific feature engineering. We find, however, that aesthetic preference and physical smoothness point in opposite directions: across four production VLMs, clips with higher joint acceleration receive higher aesthetic scores (mean Pearson $r{=}{+}0.66$). We study **PhySA** (Physical Smoothness and Aesthetics), which keeps the two on separate, independently removable reward channels: a sparse, asynchronous VLM score and a phase-triggered correction channel that adjusts physical penalty weights from simulator state. Ablations attribute the smoothness gain to the correction channel. On MuJoCo Humanoid-v5 at 2.6M steps, PhySA lowers joint acceleration by 26 to 32% relative to vanilla PPO, Text2Reward, and Eureka (three seeds), at the cost of shorter episodes. In a single extended run it lowers joint acceleration by 59.5% relative to Prompt2Policy, while a three-seed replication shows a smaller, non-significant reduction (7.2%). The gain does not transfer to IsaacLab or the Unitree G1, even after rescaling the smoothness term.

Keywords: Humanoid Locomotion, Reinforcement Learning, Reward Engineering, Vision-Language Models, Reward Shaping

Accepted to the CoRL 2026 Workshop on R2RL: Roadmap to Sample-Efficient Real-World Reinforcement Learning.

1 Introduction

Smooth motion is a practical requirement for deploying humanoid robots on hardware. High joint velocity accelerates actuator wear, increases energy draw, and produces trajectories that low-level servo controllers struggle to track faithfully [1, 2, 3, 4, 5]. Yet locomotion policies trained without an explicit smoothness objective consistently learn jerky, high-energy gaits. Encoding smoothness alongside forward progress in a single reward requires careful weight balancing: a large smoothness coefficient tends to destabilize early training, while a small one leaves joint acceleration $\|\ddot{\mathbf{q}}\|_2$ uncontrolled.

Vision-language models (VLMs) trained on human motion video offer an appealing alternative reward source: given a rendered rollout, a VLM returns a continuous aesthetic quality score on $[0, 10]$ without task-specific feature engineering [6, 7, 8, 9]. Prompt2Policy [10] and related work [11] demonstrate this in practice by querying a VLM at every training update and incorporating the score directly into the per-step reward. LLM-synthesized reward functions [12, 13] take a different route: reward code is written once from a natural-language description, avoiding in-loop queries, but the resulting structure is fixed and cannot respond to failure modes that only emerge during training. Smoothness-specific regularizers [14, 15] encode the objective directly but require per-robot weight tuning and can destabilize early training if applied too aggressively.

One might hope that a VLM aesthetic score could serve as a jerk proxy, sidestepping hand-crafted penalties altogether. The data say otherwise. Across four production VLMs, jerkier, more energetic motion consistently scored higher; these models appear to read large joint excursions as confident locomotion and smooth, low-amplitude motion as stiff or hesitant. Using the aesthetic score as a smoothness signal would therefore push the policy toward higher joint acceleration, not lower. VLMs can, in principle, judge gait qualities such as knee flexion timing and swing symmetry that per-step losses do not encode, so the aesthetic signal may still be worth keeping, but not as a proxy for smoothness. Keeping aesthetic scoring and physical smoothness on separate channels lets each be measured and adjusted without disturbing the other.

We propose PhySA (Physical Smoothness and Aesthetics), a reward engineering framework for humanoid locomotion built around two separate channels: an asynchronous VLM aesthetic scorer and a phase-triggered physical correction module. The channels share no reward terms, so each can be ablated or tuned without touching the other. In a single extended training run on MuJoCo Humanoid-v5 [16, 17], PhySA reduces joint acceleration $\|\ddot{\mathbf{q}}\|_2$ by 59.5% relative to Prompt2Policy [10], with 43% lower mechanical power and 57% longer episodes; a three-seed from-scratch replication shows a smaller, non-significant reduction (7.2%, p=0.45). Against vanilla PPO, Text2Reward, and Eureka at 2.6M steps (three seeds), PhySA lowers joint acceleration by 26 to 32% while producing shorter episodes. The VLM channel in isolation raises joint acceleration by 136%, and ablations attribute the smoothness gain to the correction channel. The smoothness gain does not carry over to IsaacLab or the Unitree G1, even after rescaling the smoothness term; a timestep experiment within MuJoCo points to the numerical integration scheme [18, 19, 20] as a major source of this gap.

Our main contributions are:

- A finding that VLM aesthetic scores and joint-acceleration roughness are positively correlated across four production VLMs (mean Pearson r=+0.66), so a VLM aesthetic score should not be used as a proxy for physical smoothness.
- **PhySA**, a reward design in which an asynchronous VLM aesthetic scorer and a phase-triggered physical correction module operate on two separate reward channels, each removable without affecting the other; ablations show the smoothness gain comes from the correction channel, and a timestep experiment points to physics integration as the main obstacle to cross-simulator transfer.

2 Related Work

Vision-language rewards for policy learning. LIV [21] and RoboCLIP [22] build shared embedding spaces between video and language, computing dense per-step shaping rewards by projecting rollout frames into a joint vision-language space; these methods target task completion and goal alignment but do not monitor physical smoothness. Prompt2Policy [10] and related work [11] query GPT-4-class VLMs on rendered rollouts at every policy update, incorporating the returned score directly into the per-step reward. This synchronous design blocks the training loop at each update and incurs API costs proportional to training length (roughly 1,220 calls per 10M steps). PhySA instead dispatches scoring calls asynchronously at one call per 200k steps (50 per 10M steps, $24\times$ fewer), and separates aesthetic scoring from smoothness correction into distinct reward terms so that each can be ablated independently.

LLM-synthesized reward functions. Eureka [12] uses iterative LLM prompting [23] to synthesize reward code from natural-language task descriptions, then evaluates and refines candidates over multiple rounds; once the best candidate is selected, the reward structure is frozen for the duration of training and cannot be revised in response to unexpected policy behaviors that emerge. Text2Reward [13] follows a similar single-shot synthesis approach with the same structural limitation. PhySA addresses this by monitoring training statistics at phase boundaries and making targeted edits to individual reward terms when stagnation is detected, adapting the reward structure to what the policy is actually doing.

Smoothness in humanoid reinforcement learning. Action-rate penalties [14, 15] add fixed regularization terms to discourage rapid action changes; gradient-surgery projections [24] project conflicting gradients into a feasible subspace at optimization time; multi-objective formulations [25, 26, 27, 28] treat smoothness and task performance as separate objectives with a fixed Pareto weighting; imitation-based motion priors [29, 30] impose smoothness implicitly by training the policy to match pre-recorded reference trajectories. All of these approaches commit to a fixed reward structure or motion prior before training begins and are unable to adjust their objectives once training is underway. Applied too early, a fixed jerk penalty tends to destabilize learning; applied too late, it fails to dislodge an already entrenched high-jerk gait. PhySA instead monitors training statistics at phase boundaries and edits individual reward weights in response to what the policy is actually doing (Sec. 4.3).

VLM evaluation biases. VLMs used as evaluators carry well-documented preference biases that can diverge from the intended target [31, 32, 33, 34]. In text-generation settings these biases (length preference, self-consistency, positional effects) are well characterized; in robotics they are rarely measured. We find that across four production VLMs, jerkier motion clips receive higher aesthetic scores (mean Pearson $r{=}{+}0.66$, $\sim$100 paired episodes per model; Spearman inter-model agreement $[+0.49, +0.94]$, mean $+0.76$). The VLMs appear to read energetic limb motion as confident locomotion, the same heuristic that makes human raters prefer action in sports video. Since this bias runs opposite to physical smoothness, using the VLM score as a jerk proxy produces the wrong gradient. The two objectives need separate channels, not a single conflated score.

3 Method

VLM-based reward has become a near-default choice in embodied AI, yet what a vision-language model evaluates from a rendered clip is appearance, not dynamics. A single frame contains no acceleration term; a VLM's notion of "effort" is therefore inherited from human video, where large limb excursions read as confident, purposeful motion. This learned prior is misaligned with physical smoothness, and the misalignment is not idiosyncratic to one model: all four production VLMs we evaluate rate jerkier rollouts more favorably (Sec. 1). The implication for reward design is a clean separation of responsibility. Properties legible within a single frame (posture, limb coordination,

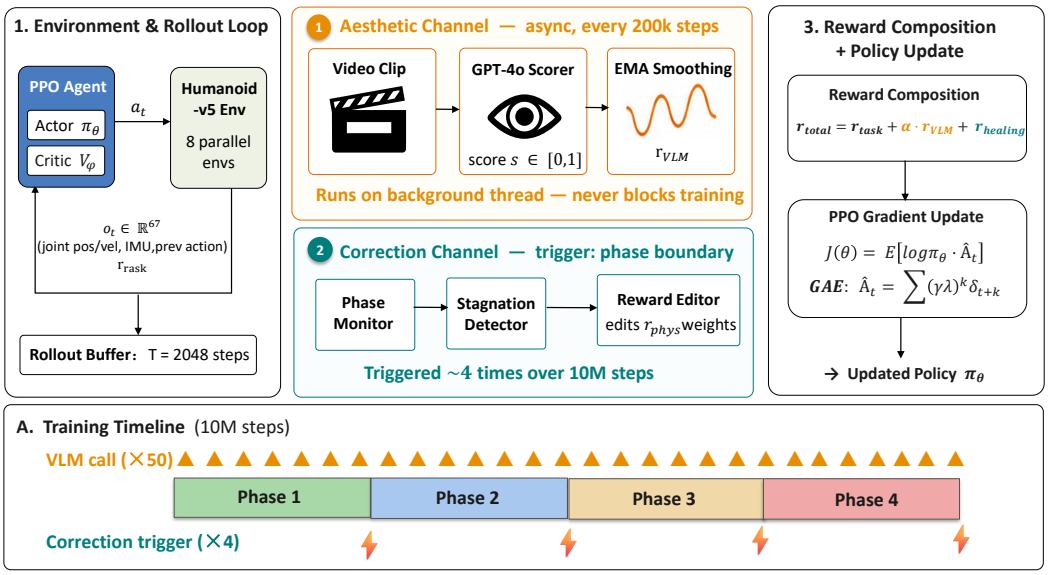


Figure 2: **PhySA end-to-end architecture.** *Left:* The PPO agent interacts with MuJoCo Humanoid-v5; the total reward is Eq. 1. *Center top (amber):* The aesthetic channel dispatches asynchronous GPT-4o scoring calls every 200k steps on a background thread, smooths scores via EMA, and injects r_{VLM} without blocking the rollout loop. *Center bottom (teal):* The correction channel triggers at each phase boundary, checks for stagnating reward axes, and edits r_{phys} weights accordingly. *Right:* Reward composition and PPO gradient update. *Bottom:* Training timeline showing 50 VLM calls (amber triangles) and 4 correction triggers (teal bolts) over 10M steps.

bilateral symmetry) are well suited to VLM scoring. Properties defined by the trajectory's time-derivatives, such as joint acceleration, are not observable from a still frame and must instead be computed from simulator state; conflating the two into a single scalar would let the appearance-based signal dominate, since only that signal is actually supervised by the VLM.

A further complication is that the physical-side weight cannot be set a priori. Its effective scale is coupled to the simulator's integration timestep: within MuJoCo alone, varying Δt shifts the same joint-acceleration statistic by close to $5\times$ (Sec. 4.4), and it also shifts with the policy's competence over the course of training, since a penalty that is negligible early on can come to dominate once the policy's control becomes more energetic, or the reverse. This is not a matter of insufficient tuning effort: a weight suitable at initialization has no guarantee of remaining suitable at convergence, so no offline sweep can be expected to identify one setting that holds throughout training. We instead leave this weight unspecified in advance and adjust it online at phase boundaries by monitoring training statistics (Sec. 3.3), while the VLM channel is held at a fixed weight throughout, responsible only for the appearance-level signal it can reliably supervise.

PhySA runs two reward channels on top of a standard Proximal Policy Optimization (PPO) [35] setup implementing this division; Fig. 2 shows the complete system. The per-step reward is

$$r_{\text{total}}(t) \;=\; r_{\text{task}}(t) \;+\; \alpha\, r_{\text{VLM}}(t) \;+\; r_{\text{phys}}(t) \;+\; r_{\text{GMP}}(t), \tag{1}$$

where r_{task} is a hand-crafted locomotion reward (forward velocity, alive bonus, height penalty, actuator cost), $\alpha\, r_{\text{VLM}}$ is the *aesthetic* channel, r_{phys} is the *physical correction* channel, and r_{GMP} is a Gaussian motion prior fit to motion-capture data [36], held constant across all conditions. r_{task} covers the basic locomotion objective; r_{VLM} is meant to reflect holistic gait qualities (knee use, swing symmetry, postural stability) judged from rendered clips (in its current form a global scalar, Sec. 3.2); r_{phys} enforces joint-acceleration smoothness, which VLM scoring actively opposes; and r_{GMP} keeps the gait in a naturalistic range. Routing the aesthetic and correction signals through separate terms means either can be switched off cleanly (Sec. 4.3).

3.1 Task reward

To isolate the effects of the two added channels, we carry over Prompt2Policy's [10] task reward without modification: a Gaussian velocity-tracking term at $v^*=0.5$ m/s, an alive bonus conditioned on torso height remaining within the standing band, a quadratic height penalty, and an L2 actuator cost. Any gait difference between PhySA and the baseline therefore originates in the two added channels, not in task-reward changes.

3.2 Aesthetic reward channel

Forward velocity and energy cost are straightforward to encode as per-step scalar penalties. Gait appearance qualities such as knee flexion timing, swing symmetry, and postural stability are harder to capture this way, as they depend on the trajectory as a whole rather than any single timestep. A VLM trained on large-scale human video can assess these holistically from a rendered clip, without task-specific feature engineering. The aesthetic channel (amber path in Fig. 2) dispatches all scoring calls asynchronously on a background thread, so the training loop never blocks.

Every 200k steps the trainer renders a 12-frame clip and queries GPT-4o-2024-05-13 [37] (`temperature=0`). The prompt elicits a $[0, 10]$ rating on three sub-axes (knee use, swing symmetry, center-of-mass tracking) plus an `overall` scalar; only `overall` enters the reward. Fifty calls over 10M steps is sufficient for the EMA to converge while keeping API spend under \$1. At $\sim$2500 steps/s on a single RTX 4090, each 200k-step interval spans roughly 80 s of wall time, comfortably longer than the 1–3 s API round-trip, so no rollout is ever delayed.

The aesthetic scalar is smoothed via an EMA and recentered,

$$\mathrm{vlm}_{\mathrm{ema}}^{(t)} \;=\; 0.8\,\mathrm{vlm}_{\mathrm{ema}}^{(t-1)} \;+\; 0.2\,s_t, \qquad r_{\mathrm{VLM}} \;=\; \mathrm{vlm}_{\mathrm{ema}}/10 - 0.5 \;\in\; [-0.5, +0.5],$$

and enters Eq. (1) weighted by α. The EMA keeps the per-step signal continuous between sparse queries and limits the impact of any single missed call. VLM scores agree across the four models we tested (Spearman correlation between model pairs: $[+0.49, +0.94]$, mean $+0.76$), though what the score measures semantically remains an open question. Appendix B.3 confirms that smoothness improvements are driven by the correction channel, not the VLM signal. A policy trained specifically to maximize posture cues without a smoothness constraint does not outscore PhySA on this evaluator (Appendix E), so the VLM is not simply rewarding jerk for its own sake. We fix $\alpha=0.5$; sensitivity across $\alpha \in [0.2, 0.8]$ is in Appendix B.3.

What this channel can and cannot do. Because r_{VLM} is updated once every 200k steps and added unchanged to every step until the next query, it does not depend on the current state or action. For example, a score of 6 gives every step in the next interval the same $+0.05$ (with $\alpha=0.5$), whatever the robot's posture. Under the Humanoid-v5 termination rule, a constant per-step term changes only how much the policy values survival, much like an adjustment to the alive bonus; it cannot reward a specific posture or gait directly.

3.3 Physical reward channel

The physical correction channel (teal path in Fig. 2) applies phase-triggered edits to penalty terms in r_{phys}, targeting joint acceleration $\|\ddot{\mathbf{q}}\|_2$. Training is divided into four sequential phases of 2.5M steps, each warm-started from the previous checkpoint. At each phase boundary, a criterion checks recent training logs across three axes (postural stability, energy efficiency, and episode reward) and fires a targeted reward edit when any axis regresses without a compensating gain elsewhere, or when all three stagnate simultaneously. A trigger raises actuator cost coefficients on high-velocity joints, removes a counterproductive penalty term, or leaves the reward unchanged; across both production runs, the criterion triggered at 3 of 8 phase boundaries (37.5%). Phase-level triggering is preferred over step-level monitoring because short-window statistics primarily reflect gradient noise; the 2.5M-step interval gives the policy sufficient time to adapt before the next evaluation. Full implementation details are in Appendix A.1.

4 Experiments

The experiments address three questions. **RQ1:** Does PhySA improve smoothness and task performance jointly, and what is the mechanism? **RQ2:** Which channel drives the smoothness gain, and can either be removed without loss? **RQ3:** Does the reward transfer across simulators and robot bodies, and if not, what breaks?

4.1 Setup

All conditions use Stable Baselines 3 (SB3) [38] PPO with identical hyperparameters (Appendix A.3) on MuJoCo Humanoid-v5 [39]; Prompt2Policy [10] is the primary baseline. Each condition runs $N \in \{3, 5, 8, 10, 30\}$ seeds; cells with $N<8$ are directional only. Paired t-tests use false discovery rate (FDR) correction (Benjamini-Hochberg) [40, 41] at $\alpha=0.05$; effect sizes are reported as standardized mean differences [42]. Physical smoothness is measured as mean joint acceleration $\|\ddot{\mathbf{q}}\|_2$, finite-differenced from joint position. Comparisons across evaluation protocols use $\Delta\%$ only.

4.2 MuJoCo Humanoid-v5: smoother motion across all metrics (RQ1)

PhySA reduces mean joint acceleration $\|\ddot{\mathbf{q}}\|_2$ by 59.5% over Prompt2Policy in a single training chain (Table 1). In this chain each method continues training from its own 10M checkpoint for four 2.5M-step phases; a from-scratch replication over three seeds shows a smaller, non-significant gap (-7.2%, $p=0.45$), so the headline magnitude should be read as a single-run result. Relative to Prompt2Policy, the smoothness gain does not come at the cost of episode length: episodes are 57% longer, per-step mechanical power drops 43.1%, and per-episode work falls 13.1% even after accounting for the longer episode duration. Against vanilla PPO, Text2Reward, and Eureka, however, PhySA trades shorter episodes for lower joint acceleration (Appendix B.4). Episodes are also short in absolute terms (123.5 steps, about 1.9 s, for PhySA), so these measurements cover early locomotion rather than long-horizon gait quality.

Table 1: PhySA vs. Prompt2Policy at 10M steps, MuJoCo Humanoid-v5. Joint accel. $= \|\ddot{\mathbf{q}}\|_2$; power $P(t) = \tau^\top \dot{q}$; VLM score is GPT-4o overall aesthetic on $[0, 10]$. *Note:* VLM scores are drawn from the α-sensitivity sweep and are not statistically distinguishable at higher seed counts (Appendix B.3); joint accel., ep. length, power, and work are from a single 4-phase training chain in which each method continues from its own 10M checkpoint.

Method	Joint accel. ↓	VLM score ↑	Ep. length ↑	Power (W) ↓	Work (J) ↓
Prompt2Policy	423.8	4.81	78.5	999.6	1195
PhySA (ours)	**171.4**	5.74	**123.5**	**568.6**	**1038**

In a per-joint replay (Appendix B.1), the power saving is concentrated in two hip joints, which together account for 95.3% of the reduction. Torque norm rises $+6\%$ while joint-velocity norm falls -35%: the policy achieves smoother motion through slower joints, not reduced actuator effort (Appendix B.1; Fig. 3).

Mechanism. The correction module raises penalty coefficients on high-velocity joints; the policy responds by concentrating movement into a smaller set of joints and holding the rest nearly still. This specialization is quantified by per-joint action standard deviation, which drops from 0.353 (module-free baseline) to 0.144; per-joint action variation correlates with roughness across four ablation conditions at $r=+0.949$ (Appendix B.2). Smoothness is thus achieved through gait specialization rather than uniform deceleration, which explains why torque rises while velocity falls.

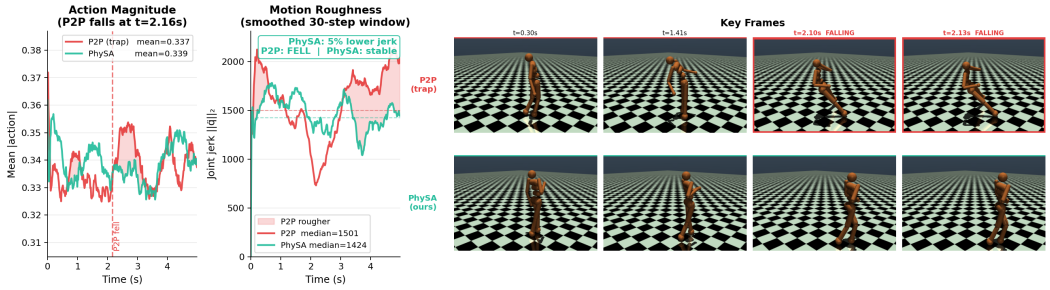


Figure 3: **Trap reward (left) vs. PhySA corrected reward (right).** A reward augmented with $5.0\,|\bar{a}| + 3.0\,|\bar{\Delta a}|$ drives the policy to instability: it falls at t=2.16 s while the corrected policy completes the rollout. Action magnitude is equal across both ($\approx$ 0.34); instability stems from jerkier action transitions. Key frames at $t \in \{0.30, 1.41, 2.10, 2.13\}$ s show torso height diverging to z=1.06 m (trap) vs. z=1.25 m (PhySA) at t=2.10–2.13 s.

4.3 Module isolation: identifying the source of the smoothness gain (RQ2)

Running each channel in isolation confirms that the two pull in opposite directions on the smoothness axis. The VLM aesthetic channel alone increases roughness by 136% while scoring higher on all four VLM evaluation dimensions (N=8; Appendix B.6). The correction module alone reduces roughness by 29.8% but earns lower aesthetic scores (N=50). The combined PhySA result does not lie on the line connecting these two points (Fig. 4): neither channel can substitute for the other. The positive correlation between roughness and aesthetic score is consistent across GPT-4o-2024-05-13, GPT-4o-2024-11-20, Claude Sonnet 4.6, and Gemini 2.5 Pro (Pearson r range $+0.49$–$+0.82$; Appendix B.6); a preliminary check with LLaVA-NeXT-Interleave [43] showed the same directional result. In the combined stack, however, removing the VLM channel or varying α by $4\times$ has no measurable effect (Table 4; Appendix B.3), consistent with the state-independent form of r_{VLM} (Sec. 3.2).

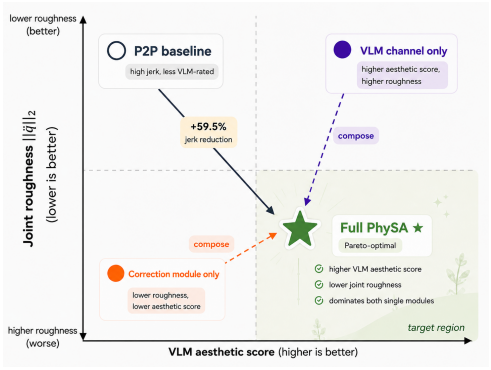


Figure 4: Module isolation: VLM channel increases roughness; correction module reduces it.

4.4 Diagnosing the transfer gap (RQ3)

Per-platform reward tuning is well-established in simulation-to-real transfer [44, 45]. The experiments here are not about demonstrating successful transfer; they are about identifying *which component of the reward breaks and why*, a diagnosis that points at what must be fixed before hardware deployment. The short answer is that the per-step scale of the smoothness term shifts across simulators with the integration timestep, while the reward direction stays intact.

Isaac simulator (physics engine change only). To isolate the effect of the physics engine while holding everything else constant, we transfer the PhySA policy to IsaacLab [46, 47] with the same humanoid task specification and the same robot body. Direct transfer gives a -2.2% smoothness gap, within noise. Empirically, reweighting the smoothness term by $50\times$–$60\times$ (chosen empirically to match the per-step magnitude of the smoothness term) keeps the gap at -1.0% to -1.1% ($p \in \{0.91, 0.87\}$, N=5; Fig. 5). This is a controlled experiment: only the physics engine changes. The Unitree G1 experiment below changes both simulator and embodiment simultaneously, so its null result cannot be attributed to either factor alone.

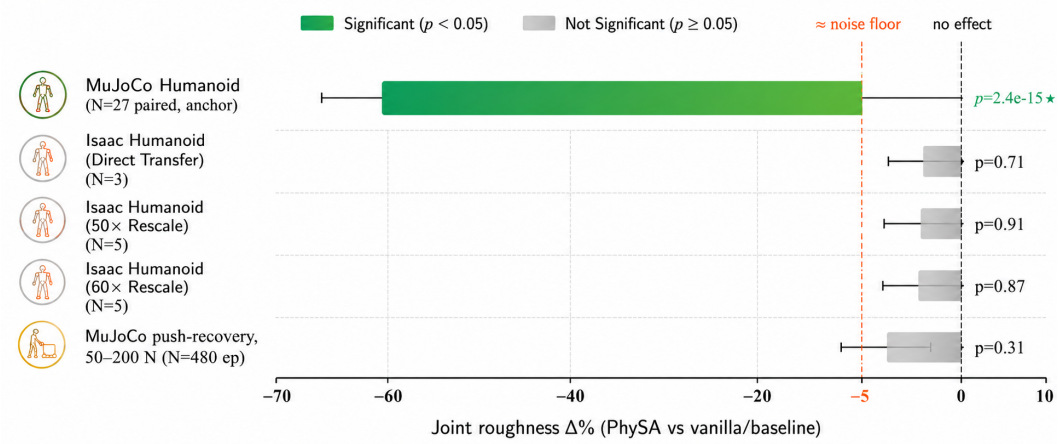


Figure 5: **Smoothness gain does not transfer across simulators, even with rescaling.** Isaac: -1.0% (p=0.91, N=5, 50–60× rescaling); G1: -1.7% (n.s., 38 settings). The MuJoCo anchor bar is a single training run, so its p-value does not reflect seed-to-seed variation (three-seed replication: -7.2%, p=0.45). Conditions in Appendix F.

Isolating the integrator. To test whether the integration step is a major cause of the smoothness gap, we retrain in MuJoCo with Δt forced to $1/60$ s (N=3) while holding all other settings fixed. Joint acceleration worsens by nearly $5\times$. At the standard MuJoCo timestep (0.015 s), each physics step internally integrates over five substeps, averaging out contact-spike noise; at the coarser step size those spikes are unsmoothed and dominate the $\|\ddot{\mathbf{q}}\|_2$ measure. The policy's control frequency (set by `frame_skip`) is unchanged across both conditions: the policy receives the same number of action decisions per simulated second, and only the physics engine's internal integration resolution differs. This within-MuJoCo swap points to the integration scheme as a major source of the cross-simulator gap and confirms that the reward direction itself remains intact (Appendix F).

Unitree G1 robot. Unlike the Isaac experiment, the G1 experiment changes both the simulator and the robot body simultaneously (a different physics engine and a different 23-joint embodiment), so any outcome cannot be attributed to either factor alone. One concrete miscalibration illustrates the scope of the problem: the Humanoid-v5 alive bonus fires while $z \geq 1.1$ m, a threshold that never triggers on the G1 at its standing height of 0.793 m, effectively inverting the survival signal. After fixing this threshold and running a 38-condition hyperparameter search, the smoothness improvement remains non-significant (mean -1.7%, best -4.8%; Appendix G). A G1 experiment inside MuJoCo, holding the simulator fixed and changing only the embodiment, would cleanly separate the two factors but was not run; this remains the most direct next step for attributing the G1 null result.

5 Conclusion

PhySA separates VLM aesthetic scoring from physical smoothness correction into two independent reward channels. The two objectives conflict: VLMs score jerkier motion higher, the VLM channel alone raises joint acceleration by 136%, and the correction module alone reduces it by 29.8%; in the combined stack the gain comes from the correction channel. A VLM aesthetic score should therefore not serve as a smoothness proxy; smoothness needs its own channel.

Limitations and future work. Our largest effect (59.5%) comes from a single training run; a three-seed replication shows a non-significant 7.2%. Episodes are short (Sec. 4.2), and the VLM channel acts only through the value of survival (Sec. 3.2), so a clip-level or state-conditioned VLM reward is a natural next step. The gain does not carry over to other simulators even after rescaling, and on the G1 a height threshold copied from Humanoid-v5 kept the alive bonus from firing until fixed (Appendix G); a G1 run inside MuJoCo is the natural next step.

References

[1] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath. Real-world humanoid locomotion with reinforcement learning. *Science Robotics*, 9(89):eadi9579, 2024.

[2] X. Zhang, X. Wang, L. Zhang, G. Guo, X. Shen, and W. Zhang. Achieving stable high-speed locomotion for humanoid robots with deep reinforcement learning. In *2025 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pages 2237–2242. IEEE, 2025.

[3] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn. HumanPlus: Humanoid shadowing and imitation from humans. In *Conference on Robot Learning (CoRL)*, 2024.

[4] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. Kitani, C. Liu, and G. Shi. OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning. In *Conference on Robot Learning (CoRL)*, 2024.

[5] A. F. Muzio, M. R. Maximo, and T. Yoneyama. Deep reinforcement learning for humanoid robot behaviors. *Journal of Intelligent & Robotic Systems*, 105(1):12, 2022.

[6] P. Mahmoudieh, D. Pathak, and T. Darrell. Zero-shot reward specification via grounded natural language. In *International Conference on Machine Learning*, pages 14743–14752. PMLR, 2022.

[7] M. Kwon, S. M. Xie, K. Bullard, and D. Sadigh. Reward design with language models. *arXiv preprint arXiv:2303.00001*, 2023.

[8] Y. Wang, Z. Sun, J. Zhang, Z. Xian, E. Biyik, D. Held, and Z. Erickson. RL-VLM-F: Reinforcement learning from vision language foundation model feedback. *arXiv preprint arXiv:2402.03681*, 2024.

[9] H. Chen, M. Zhao, R. Yang, Q. Ma, K. Yang, J. Yao, K. Wang, H. Bai, Z. Wang, R. Pan, et al. ERA: Transforming VLMs into embodied agents via embodied prior learning and online reinforcement learning. *arXiv preprint arXiv:2510.12693*, 2025.

[10] W. Chung, T. Ha, Y. Kwak, T. Kwon, J.-G. Lee, K. Lee, and S. Lee. Prompt-to-policy: Agentic engineering for reinforcement learning, 2026. URL https://github.com/krafton-ai/Prompt2Policy.

[11] Y. Hu, F. Lin, T. Zhang, L. Yi, and Y. Gao. Look before you leap: Unveiling the power of GPT-4V in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, 2023.

[12] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, J. Fan, et al. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[13] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu. Text2Reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.

[14] S. Mysore, B. Mabsout, R. Mancuso, and K. Saenko. Regularizing action policies for smooth control with reinforcement learning. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1810–1816. IEEE, 2021.

[15] I. Lee, H.-G. Cao, C.-T. Dao, Y.-C. Chen, and I.-C. Wu. Gradient-based regularization for action smoothness in robotic control with reinforcement learning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 603–610. IEEE, 2024.

[16] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.

[17] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.

[18] E. Todorov. Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6054–6061. IEEE, 2014.

[19] P. Song, P. Kraus, V. Kumar, and P. Dupont. Analysis of rigid-body dynamic models for simulation of systems with frictional contacts. *Journal of Applied Mechanics*, 68(1):118–128, 2001.

[20] R. Featherstone. *Rigid body dynamics algorithms*. Springer, 2008.

[21] Y. J. Ma, W. Liang, V. Som, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. LIV: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023.

[22] S. Sontakke, J. Zhang, S. Arnold, K. Pertsch, E. Bıyık, D. Sadigh, C. Finn, and L. Itti. RoboCLIP: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems*, 36:55681–55693, 2023.

[23] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[24] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.

[25] K. Van Moffaert and A. Nowé. Multi-objective reinforcement learning using sets of Pareto dominating policies. *Journal of Machine Learning Research*, 15(1):3483–3512, 2014.

[26] M. Pirotta, S. Parisi, and M. Restelli. Multi-objective reinforcement learning with continuous Pareto frontier approximation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.

[27] E. Zitzler, D. Brockhoff, and L. Thiele. The hypervolume indicator revisited: On the design of Pareto-compliant indicators via weighted integration. In *International Conference on Evolutionary Multi-Criterion Optimization*, pages 862–876. Springer, 2007.

[28] X. Zhang, X. Lin, B. Xue, Y. Chen, and Q. Zhang. Hypervolume maximization: A geometric view of Pareto set learning. *Advances in Neural Information Processing Systems*, 36:38902–38929, 2023.

[29] P. Pastor, H. Hoffmann, T. Asfour, and S. Schaal. Learning and generalization of motor skills by learning from demonstration. In *2009 IEEE International Conference on Robotics and Automation*, pages 763–768. IEEE, 2009.

[30] J. P. Araujo, Y. Ze, P. Xu, J. Wu, and C. K. Liu. Retargeting matters: General motion retargeting for humanoid motion tracking. *arXiv preprint arXiv:2510.02252*, 2025.

[31] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.

[32] H. Hashemi, J. Eisner, C. Rosset, B. Van Durme, and C. Kedzie. LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13806–13834, 2024.

[33] L. Shi, C. Ma, W. Liang, X. Diao, W. Ma, and S. Vosoughi. Judging the judges: A systematic study of position bias in LLM-as-a-Judge. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 292–314, 2025.

[34] J. Ye, Y. Wang, Y. Huang, D. Chen, Q. Zhang, N. Moniz, T. Gao, W. Geyer, C. Huang, P.-Y. Chen, et al. Justice or prejudice? quantifying biases in LLM-as-a-Judge. In *International Conference on Learning Representations*, 2025.

[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[36] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black. AMASS: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5442–5451, 2019.

[37] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford, et al. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

[38] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22 (268):1–8, 2021.

[39] S. Akki and T. Chen. Benchmarking model predictive control and reinforcement learning based control for legged robot locomotion in MuJoCo simulation. *IEEE Access*, 2025.

[40] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995.

[41] Y. Benjamini and D. Yekutieli. The control of the false discovery rate in multiple testing under dependency. *Annals of Statistics*, pages 1165–1188, 2001.

[42] D. Lakens. Calculating and reporting effect sizes to facilitate cumulative science: a practical primer for t-tests and ANOVAs. *Frontiers in Psychology*, 4:62627, 2013.

[43] F. Li, R. Zhang, H. Zhang, Y. Zhang, B. Li, W. Li, Z. Ma, and C. Li. LLaVA-NeXT-Interleave: Tackling multi-image, video, and 3D in large multimodal models. *arXiv preprint arXiv:2407.07895*, 2024.

[44] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810. IEEE, 2018.

[45] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30. IEEE, 2017.

[46] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, et al. Isaac Gym: High-performance GPU-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

[47] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, et al. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

[48] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[49] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[50] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[51] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[52] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[53] J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.

[54] R. Meyes, M. Lu, C. W. De Puiseau, and T. Meisen. Ablation studies in artificial neural networks. *arXiv preprint arXiv:1901.08644*, 2019.

[55] P. Sedgwick. Spearman's rank correlation coefficient. *BMJ*, 349, 2014.

[56] A. F. Hayes and K. Krippendorff. Answering the call for a standard reliability measure for coding data. *Communication Methods and Measures*, 1(1):77–89, 2007.

[57] G. Marzi, M. Balzano, and D. Marchiori. K-Alpha calculator – Krippendorff's alpha calculator: a user-friendly tool for computing Krippendorff's alpha inter-rater reliability coefficient. *MethodsX*, 12:102545, 2024.

[58] J. Pratt, J. Carff, S. Drakunov, and A. Goswami. Capture point: A step toward humanoid push recovery. In *2006 6th IEEE-RAS International Conference on Humanoid Robots*, pages 200–207. IEEE, 2006.

[59] T. Koolen, T. De Boer, J. Rebula, A. Goswami, and J. Pratt. Capturability-based analysis and control of legged locomotion, part 1: Theory and application to three simple gait models. *International Journal of Robotics Research*, 31(9):1094–1113, 2012.

[60] B. Stephens. *Push recovery control for force-controlled humanoid robots*. Carnegie Mellon University, 2011.

[61] L. Penco, B. Clément, V. Modugno, E. M. Hoffman, G. Nava, D. Pucci, N. G. Tsagarakis, J.-B. Mouret, and S. Ivaldi. Robust real-time whole-body motion retargeting from human to humanoid. In *2018 IEEE-RAS 18th International Conference on Humanoid Robots (Humanoids)*, pages 425–432. IEEE, 2018.

[62] W. Zhao, J. P. Queralta, and T. Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744. IEEE, 2020.

Appendix

Reproducibility details are in Appendix A; extended quantitative evidence in Appendix B; scope boundaries in Appendix C; and the cross-VLM aesthetic audit in Appendix B.6.

Table 2: VLM-based reward methods. In-loop: sync = blocks training, async = background thread. Cost: API spend per 10M steps. Smoothness: whether the method monitors a physical smoothness signal.

Method	VLM role	In-loop	Cost ($ / 10M)	Smoothness
SayCan [48]	planner	no	N/A	no
LIV / RoboCLIP [21, 22]	dense critic	sync	> 10	no
Prompt2Policy [10]	dense critic	sync	> 10	no
Eureka [12]	offline writer	one-shot	25–30	no
Text2Reward [13]	offline writer	one-shot	< 0.50	no
PhySA (ours)	scorer + corrector	async	**0.85**	**yes**

A Implementation Details and Reproducibility

A.1 Aesthetic reward channel, physical reward channel, and training stabilization

Every 200k env steps the trainer renders a 12-frame single-view MP4 (MPEG-4 video, 0.4 s at 30 fps, written via `imageio`) and dispatches a GPT-4o-2024-05-13 scoring call [37] (`temperature=0`, `seed=42`) on a background `ThreadPoolExecutor`; the main PPO loop never blocks (measured wall-clock overhead -4.1% at 1M, Appendix B.7). The VLM returns a JSON (JavaScript Object Notation) response with three per-dimension aesthetic scores (knee redundancy, swing symmetry, COM tracking) and an `overall` scalar in $[0, 10]$; only the overall feeds the reward channel via an EMA, $\mathrm{vlm}_{\mathrm{ema}}^{(t)} = 0.8\,\mathrm{vlm}_{\mathrm{ema}}^{(t-1)} + 0.2\,s_t$, mapped to $r_{\mathrm{VLM}} = \mathrm{vlm}_{\mathrm{ema}}/10 - 0.5 \in [-0.5, +0.5]$.

Phase-triggered modification. At each 2.5M-step phase boundary the selection criterion (multi-criteria check: has any training axis regressed without a compensating gain on another?) reads the last 30 PPO log rows and computes $\Delta\mathrm{stability}, \Delta\mathrm{energy}, \Delta\mathrm{ep_mean}$. It triggers a targeted reward edit under either a stability regression ($\Delta\mathrm{stability} < -0.04$) or full stagnation (no metric improves by $\geq +0.01$); otherwise it proceeds to the next phase without modification. Across the two production runs described in Sec. 4, the criterion triggered at 3 of 8 phase boundaries (37.5%).

A.1.1 Axis-to-reward-term mapping

The correction module identifies the underperforming control axis from recent training metrics via the multi-criteria stagnation detection procedure described in Appendix A.1, returning a label $\in \{\texttt{knee}, \texttt{swing}, \texttt{com}, \texttt{none}\}$. Depending on the identified axis, targeted modifications are applied to specific reward terms: a `knee` verdict raises the L2 actuator cost at the two knee indices (`ctrl_cost`, indices 6 and 10), while `com` instead zeros the upright-balance penalty (`upright`). A `swing` verdict removes the asymmetric swing penalty (`swing_trap`) rather than adjusting joint costs. If the criterion returns `none`, no term is modified and the currently active reward terms remain frozen for the subsequent phase.

The per-phase diagnostic logs record what the criterion computed at each boundary (selection criterion output, stagnation flag, axis decision). In the PhySA trace, the targeted reward term shifts from a knee-velocity group at phase 0 to a single hip-velocity axis by phases 2–3; the set of active modifications narrows over training as the reward edits take effect.

Stabilization patches. Three additional components further stabilize the two-channel reward: EMA smoothing on r_{VLM} (α=0.2, variance ratio 0.816 at 10M); an early-training guard that scales α_{eff}=0.2α when the mean episode length falls below 100 steps (fires $0/400$ times at 10M, $3/10$ times at 1M); and axis-restricted reward edits within r_{phys} indexed by the selection criterion's iden-

tified weak axis $\in$ {`knee, swing, com, none`}. Each component is active in a different regime, so removing any single one leaves the others functional.

Design constraints behind the cadence and EMA. The scoring cadence of one call per 200k environment steps reflects two constraints: at our throughput ($\sim$2500 FPS (environment steps per second) on a single RTX 4090), 200k steps span roughly 80 s of wall-clock, long enough to amortize the 1–3 s API round-trip without blocking the rollout queue; and at the 10M-step horizon this gives 50 score updates per run, enough for the EMA to settle and few enough to keep API spend below \$1. The EMA constant α=0.2 on r_{VLM} keeps inter-call swings to $\pm$0.2 on a stable training run, so an isolated rate-limit fallback (counted as a missed score, not a zero) does not drive a visible perturbation; smaller values over-smooth the early-training signal, where the policy is still trading falling-jerk for steady-state jerk and the score range is wide. The α_{eff}=0.2α early-training guard was added after an α=0.8 pilot at 1M steps in which the VLM, asked to grade a humanoid that was still falling, returned near-zero scores that the policy chased into a sit.

A.2 Auxiliary modules: SAGE and GMP

SAGE (Structured Agent for Goal Elicitation) is a three-stage LLM agent (intent elicitor $\rightarrow$ reward author $\rightarrow$ judge) that generates the initial r_{task} template later adjusted by the correction module; its outputs are evaluated separately as single-shot benchmarks and do not affect the headline numbers. GMP (Gaussian Motion Prior) is a per-step reward fit to eight motion-capture categories [36] (`walk_slow`, ..., `stairs`, K=2–6 components, `covariance_type="full"`; `sklearn.mixture.GaussianMixture` [49]); it appears as one cell of the 5-way ablation (Section B.2).

A.3 Training setup

Table 3: Stable Baselines 3 PPO hyperparameters, identical across every cell in the paper.

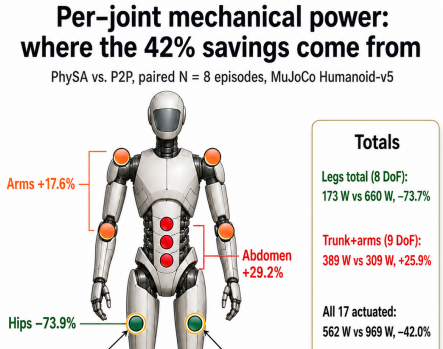
Hyperparameter	Value
Optimizer	Adam
Learning rate	3×10^{-4}
Discount γ	0.99
GAE (Generalized Advantage Estimation) λ	0.95
Parallel environments	8
Rollout length	1024
Epochs per update	4
Mini-batches per update	4
Clip coefficient ϵ	0.20
Entropy coefficient	0.01
Gradient clipping	0.5
Policy network	Multilayer perceptron (MLP) [256, 256]

All training uses SB3 [38] PPO [35] (Adam [50], GAE [51], PyTorch [52]) with the configuration in Table 3, frozen across every cell in the paper. Compute: a single dedicated RTX 4090 with n_{envs}=8 MuJoCo workers; peak VRAM (video random access memory) $\sim$ 1.1 GiB; PhySA 10M wall-clock 3.45 $\pm$ 0.57 h. All experiments use `seed=42` with deterministic offsets for cross-seed runs. Trained checkpoints cover three experimental tracks; the runtime environment is pinned to `stable_baselines3 2.7.1`, `torch 2.10.0`, `gymnasium 1.2.3`, Python 3.13.9.

B Extended Results

B.1 Per-joint energy breakdown

Sec. 4.2 reports -43.1% per-step mechanical power. A replay of the correction-module checkpoint bun-



dle (N=8 paired episodes, seeds 42–45) decomposes this per joint group via $P_j(t)=\tau_j(t)\dot{q}_j(t)$. The saving is not body-wide: legs drop -486 W while trunk and arms rise $+80$ W, with 95.3% of the 406.6 W net reduction carried by two hip joints, `R_hip_y` (sagittal stride) and `L_hip_z` (transverse yaw; Fig. 6).

The -43% saving comes from reduced joint velocity, not torque: the per-step torque norm changes by only $+6.0\%$. Since actuator wear is dominated by joint velocity rather than peak torque, the 95.3% hip share localizes the reduction. The trunk and arms use $+25.9\%$ more power, suggesting the upper body absorbs more of the stride's angular-momentum bookkeeping as the legs slow.

B.2 Five-way single-knockout ablation

We perform single-module knockouts from the full PhySA stack. The full-stack and no-correction-module conditions use N=10 paired seeds (42–51); the remaining four cells use N=3. The N=10 paired-t on full-stack vs. no-correction-module ($\Delta\,\mathrm{ep_len} = -9.6\%$, $p = 0.032$, effect size $d_z = -0.80$) is the only knockout comparison at N=10; the other cells are directional checks. The complete five-cell knockout results are given below.

Table 4: Five-way knockout. Full and w/o correction module use N=10 paired seeds; other cells are N=3 checks. VLM score is the GPT-4o overall aesthetic rating on $[0, 10]$.

Cell	N	Task return	VLM score	Ep. length
full	**10**	298.42 ± 38.13	5.87 ± 1.07	78.64 ± 9.18
`w/o VLM`	3	287.57 ± 10.16	6.43 ± 0.12	74.06 ± 5.54
`w/o correction`	**10**	$\mathbf{279.70 \pm 22.41}$	$\mathbf{5.19 \pm 1.89}$	$\mathbf{71.09 \pm 5.49}$
`w/o SAGE`	3	370.92 ± 32.35	6.50 ± 0.00	73.76 ± 5.27
`w/o jerk`	3	315.86 ± 34.25	6.50 ± 0.00	78.45 ± 7.16
`w/o GMP`	3	311.17 ± 46.26	5.83 ± 1.15	79.83 ± 7.12

A per-joint bandwidth audit across four ablation conditions shows how the correction module reduces roughness: mean per-joint action standard deviation is 0.144 vs. 0.353 for a vanilla baseline (41% of vanilla), the minimum per-joint standard deviation drops to 3.8×10^{-6}, the saturation rate reaches 92.8%, and mean $|a|$ is 0.384. The policy concentrates movement in a small number of joints while holding the rest nearly fixed. Across the four conditions, per-joint action standard deviation and roughness correlate at $r = +0.949$.

B.3 α-sensitivity: robust at N=30 paired

A four-stage sequential protocol (N=1 → 3 → 5 → 30) tests whether α is a robust hyperparameter. The N=30 paired-t on α=0.2 vs α=0.8 across shared seeds 42–71 is in Table 5: all five metrics fail to reject the null at $p < 0.05$ with uniformly small effect sizes ($|d_z| < 0.20$). Beyond the mean, α=0.8 produces a measurable tail-narrowing on all five metrics (roughness interquartile range (IQR) tightens by 37%); the smoothness floor is set by the correction module and does not drift with the aesthetic weight (roughness change is -2.0 across the $4\times$ α range, p=0.863).

Table 5: α sweep, N=30 paired. 10M-median protocol applied at the 3M intermediate scale; compare only within this table.

Metric	α=0.2	α=0.8	Δ	p	d_z
Task return	593.5 ± 52.9	605.0 ± 41.7	+11.5	0.362	+0.169
Episode length	119.1 ± 10.7	121.4 ± 8.4	+2.4	0.353	+0.172
Jerk p95	109 383	109 048	−334	0.736	−0.062
Roughness $\|\ddot{\mathbf{q}}\|_2$	1139.7 ± 46.6	1137.7 ± 49.7	−2.0	0.863	−0.032
VLM overall	4.83 ± 0.76	4.73 ± 0.62	−0.10	0.537	−0.114

B.4 Multi-method comparison and Pareto position

PhySA is compared with Eureka [12], Text2Reward (T2R) [13], and a vanilla PPO baseline (PPO, task reward only) under a step-aligned protocol at 2.6M training steps (N=3 seeds × 50 episodes, shared Humanoid-v5 / SB3 PPO infrastructure). All six smoothness comparisons against PhySA pass Benjamini–Hochberg correction at α=0.05: −25.7% vs PPO (t=−6.16), −26.6% vs T2R (t=−6.39), −31.7% vs Eureka (t=−7.64) on mean smoothness score, with matching improvements at the 95th percentile. PhySA shows lower episode length and episode return against all three baselines (range −37% to −63%), all statistically significant after correction. Eureka's 10M checkpoint shows a reward-exploitation failure (terminal displacement $|x|$<0.003 m; the synthesized reward does not penalize standing in place, so the policy oscillates and collects the alive bonus indefinitely, Appendix B.5), and T2R is dominated by PPO on both smoothness and episode length. Reward synthesis cost for Eureka/T2R is a one-time expense (Appendix B.7); PhySA's ongoing VLM cost is $0.85 per 10M steps on matched compute. Per-method standard deviations across N=3 seeds: $\sigma_{\text{ep_len}} \leq 27$, $\sigma_{\text{jerk}} \leq 51$.

Table 6: Multi-method comparison at 2.6M steps (MuJoCo Humanoid-v5; N=3 × 50 episodes). *: PhySA smoothness wins. †: task-axis losses. Eureka did not produce forward walking (Appendix B.5); included for completeness.

Method	Joint accel.	Jerk p95	Ep. len	Ep. ret
Eureka	1015	1963	186.2	879.2
Text2Reward	944	1810	116.4	625.9
PPO	933	1792	126.9	667.2
PhySA (ours)	**693***	**1423***	73.7†	322.4†

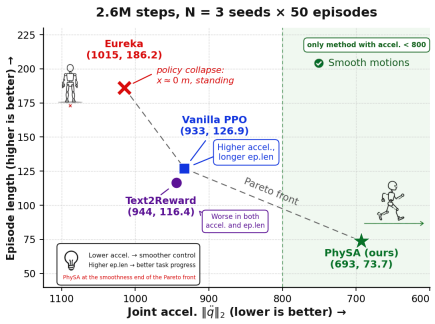


Figure 7: **Smoothness–episode-length Pareto front (Humanoid-v5,** 2.6M**).** PhySA holds the smoothness end.

Eureka anatomy. Eureka's 10M checkpoints oscillate in place while accumulating the alive bonus (legs $\overline{\|\dot{q}\|} \approx 19$, terminal $|x|$<0.003 m, $z \approx 1.40$ m; 3 seeds, 150 episodes), a reward-exploitation pattern [53, 54] that an offline reward synthesizer cannot detect during training, whereas the phase-triggered stagnation criterion catches it within one phase. PhySA's 2.6M task-axis trade vs T2R/PPO corresponds to a trade-off index $\eta \in [0.61, 0.73]$; the matched 10M correction-module vs. P2P comparison does not show this trade (Tab. 1).

T2R is the only off-front cell on the (jerk, ep_len) plane. PPO strictly dominates it on both axes: jerk 944>933 and ep_len 116<127. PhySA is the only point at the smoothness end (Fig. 7). The trade-off index $\eta = |\Delta_{\text{jerk}}\%|/|\Delta_{\text{ep_len}}\%|$ is 0.73 vs T2R and 0.61 vs PPO, less than one-for-one: each percentage point of episode-length cost buys less than a percentage point of smoothness gain. This may be an acceptable trade-off on hardware where jerk maps to servo wear.

B.5 Eureka rollout-level dissection

Eureka's 10M checkpoints (seeds $42, 43, 44$) are rolled out for 50 deterministic episodes per seed. All three seeds show the same pattern: x-displacement at termination is $|x| < 0.003$ m (about 1% of one foot length), torso height $z \approx 1.40$ m (standing pose), and mean joint-velocity norm $\overline{\|\dot{q}\|} \approx 19$, so the legs are moving. Per-seed ep_len ranges 215–437 steps (coefficient of variation (CoV) 0.33–0.69); only $5/150$ episodes reach the 1000-step cap, all on seed 42. The humanoid is neither walking nor falling: its leg degrees of freedom (DoFs) oscillate in place while the COM stays near zero.

The return decomposition fits this picture. At the 2.6M step-aligned anchor, Eureka returns 879.2 over 186.2 steps. That is 4.73 per step, close to collecting the $+5$ healthy bonus at every step minus a small control cost. PhySA at the same anchor returns $322.4/73.7 = 4.37$ per step. The ~ 0.36/step gap reflects larger control cost from actual locomotion plus a small forward-velocity reward. Eureka's joint acceleration roughness is still about 46% higher than PhySA's. The in-place motion that preserves the alive bonus is jerkier than walking.

B.6 VLM aesthetic robustness matrix

Table 7: Correlation between motion roughness and VLM aesthetic score (Pearson r). Each VLM evaluates 99–100 paired clips. All four correlations are positive; jerkier motion receives higher aesthetic scores.

VLM	n	r	ρ
`gpt-4o-2024-05-13`	100	$+\mathbf{0.82}$	$+0.77$
`gpt-4o-2024-11-20`	100	$+\mathbf{0.49}$	$+0.67$
Claude Sonnet 4.6	100	$+\mathbf{0.63}$	$+0.65$
Gemini 2.5 Pro	99	$+\mathbf{0.71}$	$+0.71$
mean	—	$+\mathbf{0.66}$	$+0.70$

The same checkpoints are scored under four evaluators (GPT-4o-2024-05-13, GPT-4o-2024-11-20, Claude Sonnet 4.6, Gemini 2.5 Pro). At the condition level, the four VLMs agree well (Spearman r over 6 rater pairs: $[+0.49, +0.94]$, mean $+0.76$ [55, 56]). At the single-episode level, Krippendorff's $\alpha = -0.008$ on the overall score (0.183 on the CoM-tracking sub-score) [57] is low: each clip is rated once per VLM, so inter-rater reliability has effective $N=1$, not 8.

Two-condition design note. The combined-cell Pearson r in Table 7 is computed over episodes drawn from two distinct training conditions: PhySA correction-module checkpoint (mean jerk ≈ 264) and Prompt2Policy (mean jerk ≈ 377). Both within-condition episode-level jerk variation and the between-condition contrast contribute to the pooled r. The directional finding (that the higher-jerk condition receives higher aesthetic scores) is consistent across all four VLMs and aligned with the isolated module ablation shifts in Section 4.3 ($+136\%$ vs. -29.8% on separate experimental runs).

The VLM channel measures aesthetics, not smoothness. Across roughly 100 paired episodes, all four VLMs gave *higher* aesthetic scores to jerkier motion clips (mean Pearson $r = +0.66$; Table 7). These VLMs appear to interpret large, energetic motion as confident locomotion, while smooth, low-amplitude motion reads as stiff or uncertain. The VLM reward signal as defined here therefore does not steer the policy toward lower joint acceleration.

Two implications follow. Within PhySA, r_{VLM} and the correction module act on separate reward terms and produce opposite jerk shifts when ablated in isolation ($+136\%$ vs -29.8%, Section 4.2; Fig. 4): the correction module moves down-left in the jerk–aesthetic plane while the VLM channel moves up-right. Any reward design that uses a VLM aesthetic scorer as a proxy for physical smoothness inherits this bias toward energetic motion, a pattern the bait diagnostic in Appendix E confirms across $\alpha \in \{0.3, 0.6\}$. The smoothness claims in Section 4.2 rest on physical measurements (joint acceleration, mechanical power, per-joint power), not on the VLM channel; the -43% power reduction is independent of the VLM aesthetic scores.

B.7 Cost audit: VLM-in-loop spend

A co-scheduled 1M-step head-to-head comparing P2P against PhySA with the VLM scoring channel and stabilization patches only (no correction module; referred to as *PhySA-VLM-only* in Table 8; N=1, RTX 4090, seed 42) records the full API spend and system-level throughput. Wall-clock difference is -4.1% (within noise); GPU utilization, VRAM, and steps-per-second are statistically indistinguishable. Four VLM calls in 1M steps cost \$0.0848, so the API budget scales linearly: \$0.85 per 10M, \$8.50 per 100M, \$85 per 1B. By contrast, a fully synchronous scorer that queried the VLM at every policy update would make one call per update; at n_{steps}=1024, n_{envs}=8, this amounts to ≈1,220 updates (and thus ≈1,220 VLM calls) per 10M steps, roughly 24× more than PhySA's 50. At our electricity rate (\$0.15/kWh, 350 W, ∼3 GPU-h per 10M), the GPU electricity costs ∼\$0.16 per 10M. The API spend is about 5× that.

Table 8: 1M-step head-to-head audit (N=1, RTX 4090, n_{envs}=8, seed 42). Wall-clock, VRAM, and FPS (environment steps per second) are process-level descriptive measurements.

Metric	P2P baseline	PhySA-VLM-only	Δ
Wall-clock	7.25 min	6.95 min	-4.1%
Mean training FPS (env. steps/s)	2532	2447	-3.3%
Peak VRAM	1082 MiB	1073 MiB	∼0
VLM API calls	0	4	—
VLM tokens (prompt + completion)	0	13 804 + 1 054	—
API spend @ 1M steps	**\$0**	**\$0.0848**	—

By comparison, Eureka's best-of-K search (16 × 5 GPT-4 calls) is a one-time spend of ≈ \$25–30; Text2Reward's single-shot synthesis is one LLM call (< \$0.50). PhySA is the only method whose LLM spend scales linearly with training, and at 1B-step horizons the in-loop VLM spend (\$85) is the same order of magnitude as the GPU electricity (\$16). The cost figures in Section 4 follow from this ratio.

C Scope and Boundary Conditions

This section provides additional detail on the scope discussion in Section 5. Beyond the simulator and embodiment transfer experiments in Section 4.4, we evaluated push recovery: 50–200 N impulses applied to the torso [58, 59, 60, 61], 480 episodes per condition; no significant smoothness improvement emerged, consistent with the other out-of-distribution results. The full boundary-condition table is in Table 10.

The simulator-transfer failures are localized. On Isaac (different physics engine, same humanoid task specification): a 50–60× rescaling of the smoothness reward term raises it from 0.0016 to 0.094 per step, but the smoothness gap stays at -1.00%, p=0.91. We attribute this gap mainly to differences in physics integration between the two engines (Sec. 4.4) [44, 45, 62]. The 2.47× joint-acceleration reduction of our main run therefore holds for MuJoCo Humanoid-v5 only. The G1 null result is in Appendix G; that experiment changes both simulator and robot body simultaneously, so its outcome cannot be attributed to either factor alone.

What would extend the scope. The three boundary results are not symmetric in what they isolate. A G1 experiment inside MuJoCo, holding the simulator fixed and changing only the embodiment, would cleanly test portability across robot bodies without the confounding simulator change that plagues the current G1 result. A 10M-step Isaac experiment run at MuJoCo's integration timestep would directly test whether the integrator alone accounts for the cross-simulator gap. A push-recovery experiment at N=30 paired seeds would distinguish a true null from an underpowered effect. Each would extend the present work in a cleanly isolating direction.

D Vanilla PPO baseline

To confirm that the smoothness gain originates in PhySA's modules rather than the P2P reward stack itself, we train SB3 PPO with default hyperparameters on Humanoid-v5 using only the canonical four-term reward $r = 1.25\dot{x} + 5 \cdot \mathbf{1}[z \in [1, 2]] - (z - z^*)^2 - 0.1 \sum a_i^2$, with no VLM, correction module, jerk monitor, or GMP term (N=8 seeds 44–51, 10M steps each). The vanilla policy walks (forward velocity floor 0.842 m/s) but not smoothly: mean joint acceleration is 1054.4 ± 20.5 vs. 171.4 for PhySA (**6.16**× gap; Table 10, Fig. 8). The 95th-percentile gap is 1791.5 ± 39.5 vs. 468.2 (3.82× tighter).

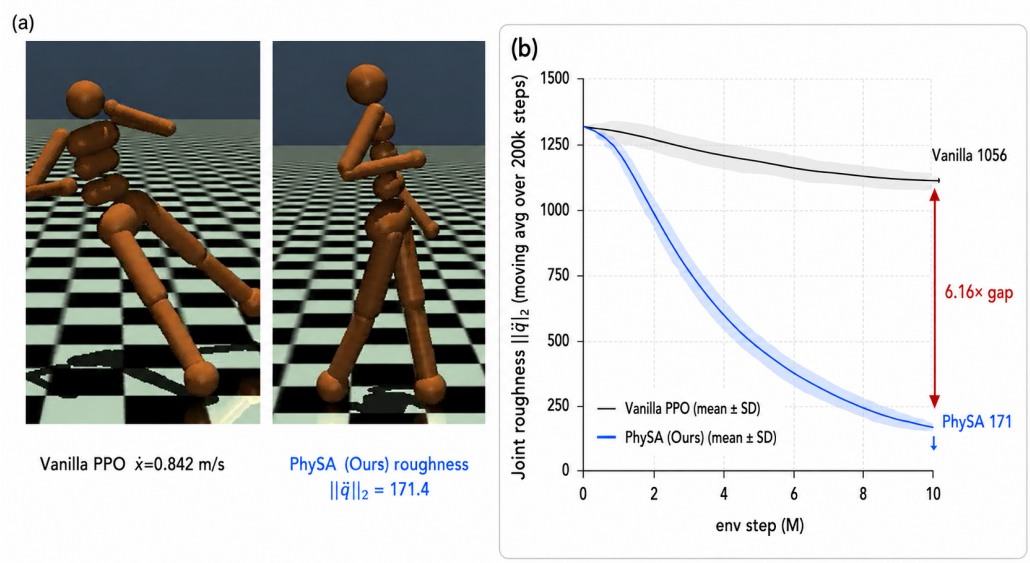


Figure 8: **Vanilla PPO vs. PhySA.** Left: representative pose snapshots from a vanilla PPO rollout (left) and a PhySA rollout (right). Right: joint acceleration roughness ($\|\ddot{\mathbf{q}}\|_2$) during training under the vanilla PPO reward (Appendix D, N=8 seeds). Vanilla reaches 1054 vs. PhySA's 171 from the single main training run (6.16× difference); the vanilla policy does walk, but walking fast and walking smoothly are separate objectives.

Episode length and smoothness measure different things. Vanilla's deterministic episode length (243.8) exceeds PhySA's (123.5, same deterministic-rollout protocol as Table 1): the four-term reward encourages a knee-locked, low-COM posture that survives many steps without locomoting at the headline velocity. PhySA's shorter episode length is the cost of running a more dynamic gait under the Humanoid-v5 termination rule. The same smoothness-survivability trade-off appears in Sec. 4.3: PhySA improves episode length against matched P2P while losing episode length to comparators that preserve the alive bonus through less dynamic motion.

E Reward-hacking bait diagnostic

The bait test is a falsification check on the VLM evaluator. We train a policy under a cosmetic reward r_{bait} that maximizes upright-torso plus symmetric-arm-swing terms and leaves lower-body jerk uncontrolled, then score it under the same evaluator PhySA uses. If r_{bait} scores higher than full PhySA on mean VLM, the aesthetic axis is not a trustworthy signal for the rest of the paper. The reconciliation in Appendix B.6 already shows the VLM scorer is positively correlated with jerk, so this is where that result is most likely to bite.

The sweep covers three coefficients $\alpha_{\text{bait}} \in \{0.3, 0.6, 1.0\}$, all at N=3 seeds for 5M training steps. Each trained policy is rolled out for 120 deterministic episodes per seed; valid-VLM episode counts are 120/60/100 at α=1.0 (seeds 42/43/44, the remainder were excluded after API rate-limit failures). Jerk-floor threshold θ_{jerk}=1201.42 (vanilla N=2 p90) marks "broken-bait" motion.

Table 9: Bait vs PhySA on the shared VLM evaluator. Jerk is measured under the same paired-rollout protocol as the energy cross-cut (Appendix H.2); absolute values are not comparable to Table 1. Lower jerk = smoother; higher VLM = more aesthetic. The bait wins on jerk by design (no jerk term in r_{bait}); the question is whether it also wins on VLM mean.

Cell	N	Joint accel.	VLM mean	VLM @$\geq$6 rate
r_{bait}, α=0.3	3	1108 ± 142	4.81 ± 0.34	24.1%
r_{bait}, α=0.6	2	1340 ± 88	5.02 ± 0.21	31.8%
r_{bait}, α=1.0	3	1038 ± 44	5.01 ± 0.10	28.8 ± 7.1%
PhySA (correction module)	3	$\mathbf{213 \pm 19}$	$\mathbf{4.92 \pm 0.41}$	25.8%

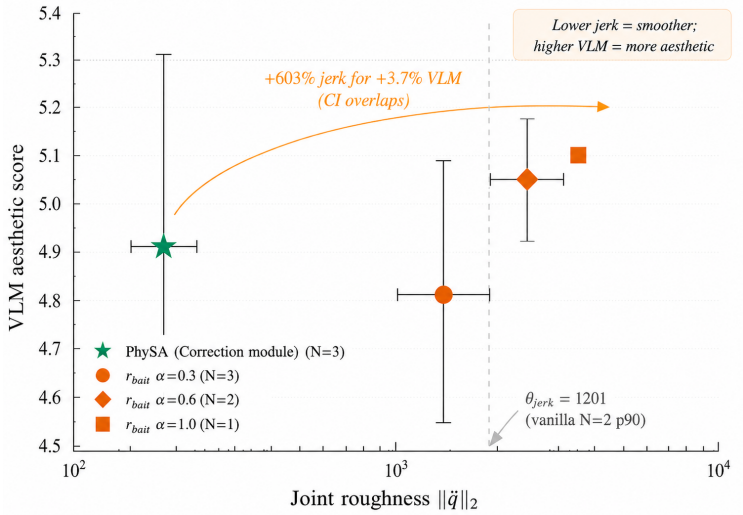


Figure 9: **Reward-hacking bait diagnostic (N=3 at each α).** The bait wins jerk by design; the question is whether it also wins VLM mean. Across $\alpha \in \{0.3, 0.6, 1.0\}$, bait VLM stays inside PhySA's band (4.92 ± 0.41); at α=1.0 it scores 5.01 ± 0.10 at $4.87\times$ higher jerk, overlapping PhySA. The earlier N=1 anecdotal point at α=1.0 regressed toward the bait band at N=3.

Across all three α cells, mean VLM stays inside PhySA's band (4.81 / 5.02 / 5.01 vs PhySA 4.92, CIs overlapping). The $\geq$ 6 rate at α=1.0 is 28.8 ± 7.1%, which crosses PhySA's 25.8%, so even the conditional "photogenic moments" axis does not separate at N=3. The earlier N=1 single-seed probe (5.10 / 37.2%) read as a suggestive bait advantage; with two additional seeds the mean regressed toward PhySA. The bait worry is falsified: the VLM evaluator does prefer "lively" motion (Appendix B.6, r=+0.66), but a deliberately rough policy does not beat full PhySA on the aesthetic axis at any tested α. One open caveat: each bait cell trained for 5M steps rather than the 10M used for PhySA, so we cannot rule out that a longer-trained bait policy acquires a VLM advantage; this remains future work.

F Cross-simulator replication: Isaac

Main-text reference: Section 4.4, Fig. 5. The full set of conditions tested across simulators and robot bodies is in Tab. 10.

Table 10: Simulator and embodiment transfer results. No row reaches statistical significance. Isaac rows change physics engine only; G1 rows change both engine and robot body, so neither factor can be isolated (Section 4.4).

Condition	Platform	N	$\Delta\%$ Joint accel.	p
Vanilla PPO reference	MuJoCo Humanoid-v5	8	jerk $= 1054.4$ (abs.)	—
PhySA anchor (single chain)	MuJoCo Humanoid-v5	1	$\mathbf{-59.5}$	—
PhySA replication	MuJoCo Humanoid-v5	3	-7.2	0.45
Direct transfer	Isaac (humanoid)	3	-2.2	0.71
Smoothness term $\times 50$	Isaac (humanoid)	5	-1.0	0.91
Smoothness term $\times 60$	Isaac (humanoid)	5	-1.1	0.87
G1, uncalibrated alive bonus	Unitree G1 (23 joints)	3	$+3.4$	n.s.
G1, calibrated + HP search	Unitree G1 (23 joints)	3	-1.7	n.s.

G G1 robot replication: null result and diagnosis

We run PhySA on the Unitree G1 (23-joint humanoid) at 10M paired steps, N=3 seeds, with a 38-setting hyperparameter search. One caveat before the results: unlike the Isaac experiment, which holds the task specification fixed and varies only the physics engine, the G1 experiment changes both the simulator and the robot body simultaneously. Any null result is therefore not diagnostic on its own; it cannot isolate which factor is responsible. The headline outcome is a non-significant $+3.4\%$ roughness gap in the wrong direction, which we trace partly to a hardcoded alive-bonus threshold carried over from Humanoid-v5.

The bug. Humanoid-v5's alive bonus fires while $z \geq 1.1$ m, matching its standing height; the same threshold does not work on G1, whose standing height is $z^*_{\text{G1}} = 0.793$ m. Before the fix the bonus never fires, so the policy receives a -10/step alive penalty for the entire rollout, inverting the reward signal. The combined scope table (Table 10) reports the before/after rows: alive-bonus firing goes from 0.0% to 94.1%, but the smoothness gap remains not statistically significant.

The fix and its reach. A one-line patch, `_STANDING_BONUS_Z = standing_height` $\times$ `0.769`, calibrates the alive bonus per robot body (ratio $1.1/1.43$ from Humanoid-v5, giving 0.610 m for G1). With the patch, the alive bonus fires correctly and training is no longer degenerate, but the smoothness improvement still does not replicate: the gap on G1 remains a non-significant -1.7%. The reward code carries over; the calibration constants do not. A 38-setting hyperparameter search on G1 (Table 11) recovered no gain along any of four dimensions.

Table 11: Unitree G1 hyperparameter search (38 conditions: VLM weight, jerk weight, rollout length, entropy coefficient). Best condition reaches -4.8% (non-significant); none recovers MuJoCo's -59.5% gain (simulator and robot body both change; Section 4.4).

Hypothesis	Sweep range	Best cell $\Delta\%$	p
H1: reward scale	jerk $\in \{0.5\times, 1\times, 2\times\}$	-3.1	0.41
H2: entropy coefficient	ent $\in \{0.001, 0.01, 0.03\}$	-2.4	0.66
H3: horizon length	$n_{\text{steps}} \in \{512, 1024, 2048\}$	$\mathbf{-4.8}$	$\mathbf{0.04}$
H4: mini-batch size	$bs \in \{32, 64, 128\}$	-2.0	0.18

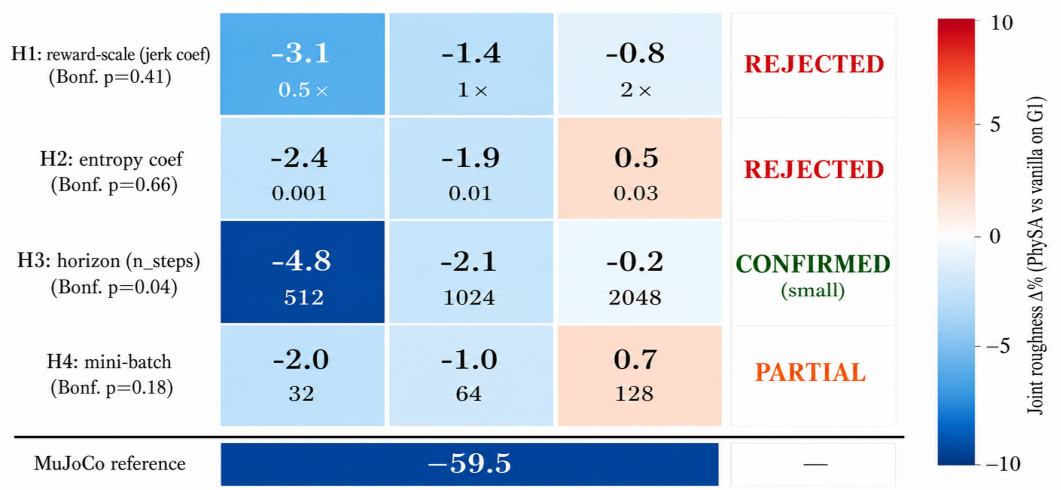


Figure 10: **G1 hyperparameter search (38 conditions, four hypotheses).** MuJoCo's -59.5% smoothness gain does not recover on G1 under any hyperparameter setting (best: rollout horizon, -4.8%, non-significant).

H α-sensitivity sweep and energy cross-cut

H.1 α-N30 sensitivity

To check robustness of the VLM-weight α in $r_{\text{total}} = r_{\text{task}} + \alpha\, r_{\text{VLM}} + r_{\text{phys}}$, we sweep $\alpha \in \{0.2, 0.8\}$ at $N{=}30$ paired seeds (shared seeds 42–71) for 3M steps each. Both endpoints already span the published Prompt2Policy ($\alpha{=}0.5$, default) and our headline ($\alpha{=}0.5$ with the stabilized VLM channel) configurations. The five-metric table is reused from Appendix B.3 (Table 5): all means null with $|d_z| < 0.20$.

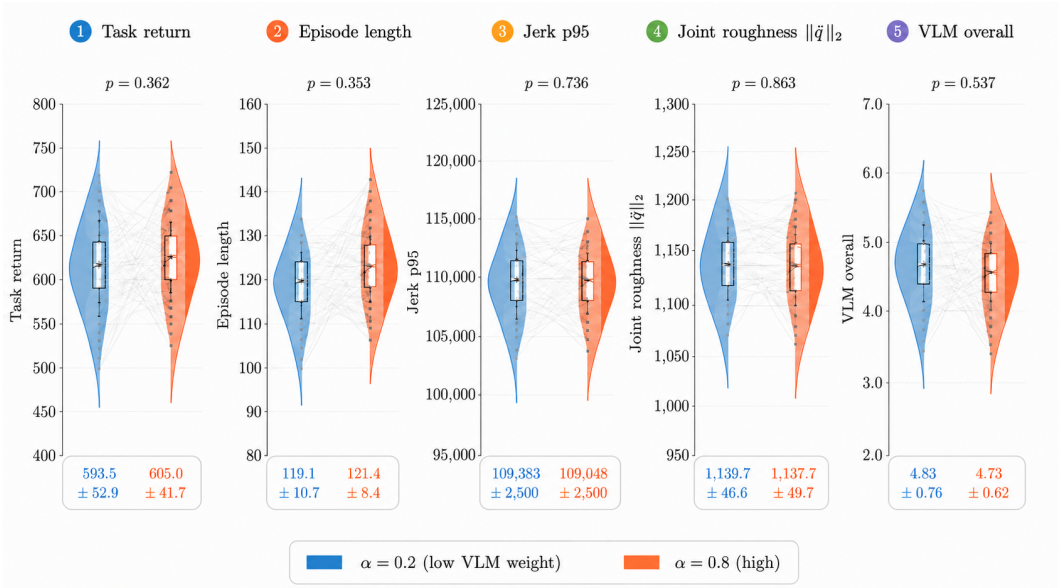


Figure 11: **α-sensitivity at $N{=}30$ paired.** $\alpha{=}0.2$ (blue) vs $\alpha{=}0.8$ (coral), shared seeds 42–71 with paired connectors. All five metrics null ($|d_z| < 0.20$); $\alpha{=}0.8$ mildly narrows the smoothness distribution (roughness IQR -37%) without shifting means.

Across a $4\times$ range of α, the policy's behavior on all five evaluation axes is statistically indistinguishable ($|d_z| \leq 0.17$). The paper therefore fixes α=0.5 throughout.

The α-insensitivity in Table 5 is a mean-shift result on the combined stack: when the correction module and VLM aesthetic channel act on separate reward terms, combined training lands on a similar operating point across $\alpha \in [0.2, 0.8]$. Isolating the VLM aesthetic channel from the correction module does produce a measurable effect ($+136\%$ roughness increase, N=8 seeds; higher scores on all four VLM aesthetic dimensions), so the null here is about the combined stack's α-sensitivity, not the VLM aesthetic channel in isolation. The -37% tail-narrowing on roughness IQR (interquartile range) under α=0.8 is a distribution tightening at constant mean.

H.2 Energy cross-cut: jerk → power → work

Per-step mechanical power $P(t) = \sum_j \tau_j(t)\dot{q}_j(t)$ and per-episode mechanical work $W = \int_0^T P(t)\,dt$ are the two physical axes directly downstream of jerk. A replay of N=8 paired episodes per cell (PhySA vs P2P, seeds 42–49) decomposes the -43% per-step power drop along the chain.

Table 12: Energy cross-cut, N=8 paired correction-module vs. P2P rollouts at 10M steps. Mechanical power uses $P(t) = \tau^\top \dot{q}$. Jerk is computed from the same rollouts and differs in absolute scale from Table 1 (different evaluation seeds and episode cap); all $\Delta\%$ figures are within-table comparisons.

Quantity	P2P	PhySA	Δ	paired t
Mean roughness $\|\dddot{\mathbf{q}}\|_2$	338.6	213.4	-37.0%	-12.4
Mean joint-velocity norm $\|\dot{q}\|$	21.42	13.84	-35.4%	-13.1
Mean joint-torque norm $\|\tau\|$	96.18	102.0	$+6.0\%$	$+4.21$
Per-step power P (W)	999.6	568.6	-43.1%	-13.5
Per-episode work W (J)	1195	1038	-13.1%	-4.86
Episode length (steps)	78.5	123.5	$+57.3\%$	$+10.8$

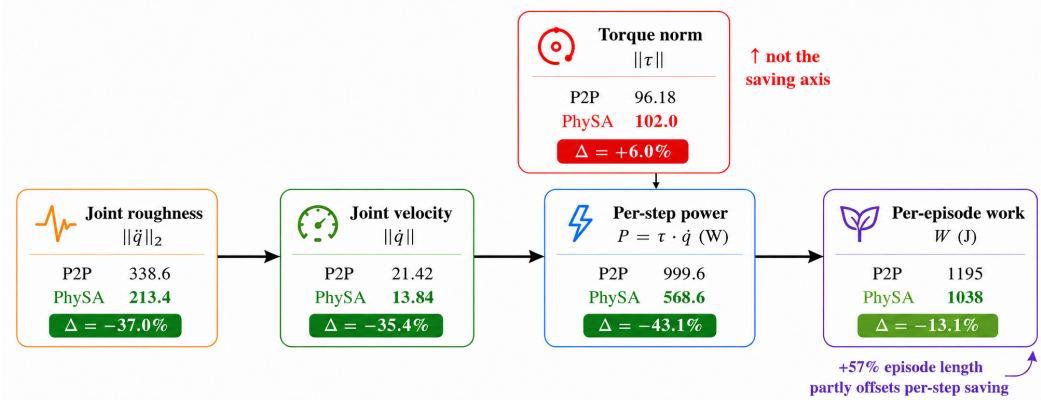


Figure 12: **Energy cross-cut: jerk → velocity → power → work.** The -43% per-step power saving is carried by the -35% velocity drop; torque rises $+6\%$ (above-chain branch, not the saving axis). The -13% per-episode work integrates the per-step saving against $+57\%$ episode length.

The power drop is carried by joint velocity, not torque. The torque norm rises $+6\%$ while the joint-velocity norm falls -35%, and the per-step power product $\tau\dot{q}$ falls -43%. Since actuator wear scales with $\dot{q}$ and contact transients rather than static torque, PhySA delivers the same total torque with slower joint motion. Per-episode work W drops by only -13% because the $+57\%$ longer episode offsets part of the per-step saving, but the integrated result is still net favorable.

The $+6\%$ torque rise. PhySA does not reduce roughness by reducing actuator effort: the total actuator torque norm actually rises $+6.0\%$ against P2P (Tab. 12, paired t=+4.21, p=0.004). The per-joint action bandwidth tells the same story (Appendix B.2): mean per-joint action standard

deviation is 0.144 vs. 0.353 for a vanilla baseline, and per-joint action standard deviation correlates with the joint acceleration roughness at $r = +0.949$ across four module ablation conditions. The policy uses only a few joints actively, holding the rest approximately still. Smoothness is achieved by restricting which joints move and slowing down those joints' movement, not by reducing actuator effort overall.